Predicting League of Legends Match Outcomes

1. Introduction

League of Legends is a multiplayer online battle arena (MOBA) video game developed by Riot Games, wherein players are divided into two teams of five and must strategize in order to take objectives and win. While this is a relatively simple concept, the problem lies in the overwhelming amount of game mechanics at play in determining the outcome of a match. There isn't a single 'telling' variable, but rather a combination of factors such as player kills and gold earned throughout the duration.

This project is motivated by the idea that the match outcome for any given team can be predicted based on the performance of its five individual players. Like any other sport or e-sport, Riot hosts the League Championship Series, where professional teams from different regions compete for the title of LCS winner. As a viewer of these tournaments, if you can predict which team will win then you can make strategic bets to profit financially. Arguably more importantly, you can cheer for the correct team and say "I told you so" to everyone who doubted your conviction.

The findings suggest that while it is possible to predict match outcomes given post-game statistics, the specific data and methodology used in this project is not generalizable to lower-ranked games with the same accuracy. Additionally, further work can be done to use timeline data as an early-game predictor, which better serves the motivation to predict the outcome for an *ongoing* match.

2. Related Works

Academic work and research on this topic typically employs one of two approaches, predicting a match outcome using either player match history or the current game's statistics, whether at the individual player or team level.

A recent publication to the 2024 IEEE International Conference on Computing (ICOCO) found that by analyzing individual player's past overall win-rate, role win-rate and champion win-rate, a match's outcome can be predicted with 97.5% accuracy using a combination of gradient boosting, logistic regression and deep neural networks.[1] This is an important, key finding because it is an example of making predictions without needing to access current match statistics.

3. Dataset & Featurization

The dataset chosen for this project was found on Kaggle and contains data from over 20k matches. Each match consists of 10 rows, five rows per team, one for each individual player, resulting in over 200k total rows. In terms of features, the dataset originally contained over 100 columns, ranging from identifiers (server, team ID), to player scores (KDA, gold, vision), team scores (win, objectives taken) and item details.

To begin featurization, the number of features was reduced to just 14. This included match and team IDs, player scores and team scores. Data such as server location and item details were removed from the final lineup because they have little to no correlation with a match's outcome.

Additionally, analysis was done to identify and remove some specific features, which were labeled as "spoilers". These were stats like "team_tower_kills", which wouldn't necessarily be known at earlier points in the game and have a high correlation with match outcome as shown in figure 1, such that their inclusion would skew the final model's performance higher than is accurate.

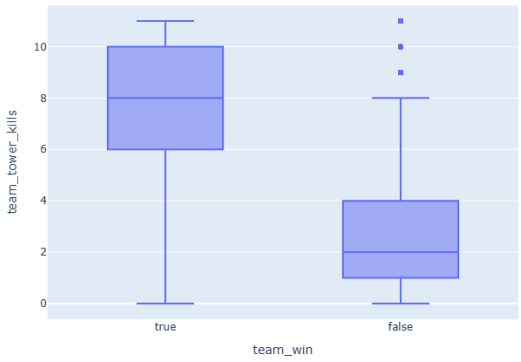

Figure 1: distribution of team_tower_kills by team_win

Because the data was collected at the individual player level and this project aims to predict outcomes at the team level, the data was flattened into two rows per match, one for each

team. Each player on a team is in one of five roles: top, jungle, middle, bottom or support. One goal of this project was to identify which variables are "most important" in determining the outcome. For example, this might mean that the kills secured by mid-lane makes more of an impact than bot-lane. For this reason, the flattened data differentiated between roles to maintain this relationship (top_kills, mid_gold, etc...).

One aspect of the original dataset is that all scores were collected post-game. This is a problem for several reasons, most glaringly because the values for each team are aggregates of scores collected throughout the whole game. This means that values are almost entirely determined by the match duration. Take gold for example, one of the most important resources in a game. It's expected that a team will win a 15 minute match with less total gold than a team in a 45 minute match. This relationship is shown in figure 2. Additionally, the model is only useful if one can apply it to an ongoing match for which the outcome isn't yet clear. For this reason, the data was normalized to create a second, separate dataset with values scaled based on match duration. This way, values are saved as per-minute and can therefore be compared to the statistics of any ongoing match.

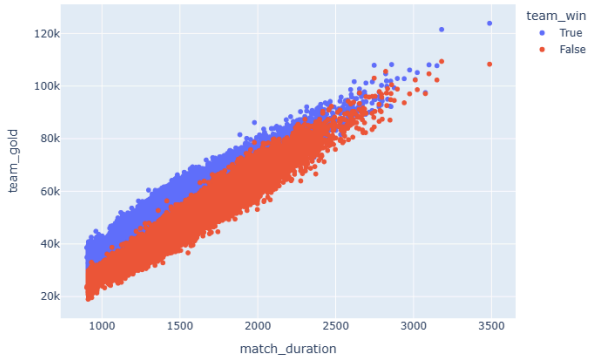

Figure 2: team gold as a function of match duration, where color represents match outcome

Finally, it was important to consider that the original dataset was created using games from the Challenger rank, which represents the top ~0.03% of players globally. This means that the model will be trained on a very specific subset of players, and therefore might not generalize well to the average game. Because of this, a second dataset was created containing 61 of my own past matches, scraped using the Riot Games API, which could be used as a test for "generalizability" of the final model. The dataset was featurized and a separate normalized version was created in the same way.

4. Model Training & Testing

Now that the training dataset was finalized, the final models could be created. Given the textual, non-sequential nature of the data, simple deep neural networks were appropriate for this project. They would be trained as a binary classifier for the "team_win" label (predicting either TRUE or FALSE). Because the data was split evenly (50/50 on win/loss), the models could be evaluated using percent accuracy.

First, each of the datasets (original and normalized) were split into 70%, 15%, and 15% for training, testing and validation. Next, the two models were created through hyperparameter tuning to identify the highest accuracy architecture for each. This process tested different parameters for units per layer, learning rate (α), batch size, number of epochs and dropout rate.

Model 1, trained on the original data, had a validation accuracy of 96.12% while model 2, trained on the normalized data, had a validation accuracy of 96.42%. Each model had three dense layers and one sigmoid activation layer. Their final architectures are shown in Table 1 below.

model	layer units	α	batch size	epochs
1	128, 32, 32	0.001	64	15
2	64, 64, 64	0.01	64	15

Table 1: Final model architectures

One interesting finding from the model training is that architectures with dropout layers consistently performed worse than those without, rarely, if ever, reaching 90% accuracy. This implies that either the model was not overfitting on the original data, or the model was small enough that adding dropout layers reduced its ability to successfully learn patterns.

5. Results

After the models were created, they were further tested on my dataset of low-rank games. Model 1 had an accuracy of 86.07% on the raw data while model 2 had an accuracy of 89.34% on the normalized data.

Besides the model's ability to correctly predict match outcomes, this project was interested in identifying which player or team scores have the biggest impact on the outcome. The distribution of feature importance for each model is shown below in figure 3. The color and spread of a feature indicates whether (relative) high values for this feature have a positive or negative impact overall.

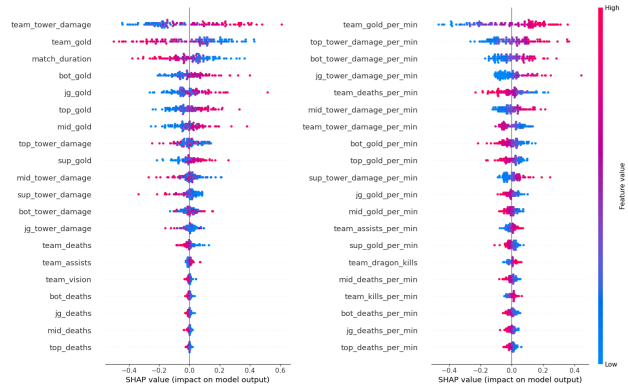

Figure 3: SHAP feature importance values, model 1 is on the left and model 2 is on the right

This figure is an example of identifying feature importance by calculating Shapley values. This is a method of AI interpretability which uses aspects of game theory to calculate how each feature changes a row's prediction away from the average. This can be done by temporarily removing a feature to gauge its importance through how the model performs without it.

6. Discussion

The increased accuracy on the normalized data in both training and testing shows that per-minute statistics can be more accurate indicators of a match outcome than post-game scores. Of course, this isn't made fully apparent because all datasets used in this project are post-game, but it follows that model 2 would perform better on an ongoing match.

Finally, the results show that a match's different scores have varying impact on the outcome. Looking at the feature importance of model 2, we can see that gold_per_minute is the most important feature, which makes sense because this is the currency earned through other scores such as kills, assists and objectives. Also, we can see that the scores of players in specific roles such as top and bottom matter more than those in more assistant roles like support. It tracks that top_tower_damage is an important feature, because this is an isolated lane. Once a player in the top lane has established dominance, they are able to easily take further objectives without much counterplay.

Additionally, it's interesting that map objectives such as dragon kills and baron kills, creatures that when killed give a team gold, XP and buffs, aren't shown to be important compared to individual player contributions.

7. Limitations

This project faced several limitations as a result of the chosen dataset, which contains only post-game statistics and comes from high-skill lobbies. The model's lower accuracy during testing is unsurprising given the difference in skill between datasets. The results highlight the importance of choosing a representative training dataset.

More limitations arise about the low-skill dataset created by scraping my own games. First, the dataset was small because of Riot API's call limits and short time range of saved games. This meant that I could only scrape the few dozen matches I've played recently, within the past year. Additionally, the fact that I appear in all of these games means it isn't random.

8. Future Work

Work can be done to improve similar projects in the future by collecting data from all skill levels and testing more varied model architectures. A balanced dataset across all ranks would ensure a more generalizable final model, while testing more model architectures with varying number of layers and more methods of regularization could improve accuracy.

Finally, it would be interesting to repeat this project with timeline data (data taken at specific points in the match). This might mean a dataset of player scores at the 10 minute mark, along with their match outcome.

References

[1] C. J. A. Cordova, C. V. A. Villaceran and C. F. Peña, "Predicting League of Legends Match Outcomes Through Machine Learning Models Using Past Match Player Performance," 2024 IEEE International Conference on Computing (ICOCO), Kuala Lumpur, Malaysia, 2024, pp. 522-527, doi: 10.1109/ICOCO62848.2024.10928259